NIFTY 100 FINANCIAL INTELLIGENCE PLATFORM

Comprehensive Analyst & Technical Developer Guide

---

1. Executive Summary & Architecture

The Nifty 100 Financial Intelligence Platform is an enterprise-grade financial analytics suite built for equity research analysts, portfolio managers, and quantitative researchers. It ingests 10 years of balance sheets, P&L; statements, cash flows, stock prices, and qualitative analysis across 92 companies in the Nifty 100 index.

The platform integrates dynamic SQLite storage, an automated financial ratio engine computing 50+ KPIs, stock screener with preset filters, peer percentile benchmarks, machine learning KMeans clustering archetypes, NLP text parsing, automated 2-page PDF tearsheet generation, an 8-screen Streamlit web application, and a 16-endpoint FastAPI REST API server.

Key System Specifications:

- Database Engine: SQLite 3 (`data/nifty100.db`) with Foreign Keys enabled
- ETL Ingestion Pipeline: Pandas & OpenPyXL loader with 16 Data Quality (DQ) validation rules
- Analytics Engine: 50+ financial ratios, 6 CAGR edge-case handlers, CFO quality scoring
- Machine Learning Engine: Scikit-learn KMeans (5 archetypes) with sector median imputation
- Interactive Dashboard: Streamlit 8-screen UI running on `localhost:8501`
- REST API Backend: FastAPI asynchronous ASGI server running on `localhost:8000`
- Automated Reports: ReportLab 2-page company tearsheets, sector PDFs, and portfolio summary

2. Data Ingestion & ETL Architecture

The ingestion pipeline (`src/etl/loader.py`) reads 12 Excel source files from `data/raw/` and `data/supporting/` into 10 structured SQLite tables. All tickers and years undergo strict normalization (`src/etl/normaliser.py`).

Master Database Schema Summary:

Table Name	Primary Key	Foreign Keys	Description / Record Count
companies	id (Ticker)	None	92 master company profiles with industry tags
sectors	company_id	company_id -> companies	Broad and sub-sector industry mapping
profitandloss	(company_id, year)	company_id -> companies	10-year Income Statement history (~1,073 rows)
balancesheet	(company_id, year)	company_id -> companies	10-year Balance Sheet history (~1,140 rows)
cashflow	(company_id, year)	company_id -> companies	10-year Cash Flow Statement history (~1,056 rows)
financial_ratios	(company_id, year)	company_id -> companies	50+ computed KPIs across 1,155 company-years
peer_groups	(company_id, peer_group_name)	company_id -> companies	11 peer industry group mapping tables
peer_percentiles	(company_id, peer_group_name)	company_id -> companies	Percentile ranks (0.0 to 1.0) across 10 KPIs
stock_prices	(company_id, date)	company_id -> companies	5,520 daily stock price records
market_cap	(company_id, year)	company_id -> companies	Valuation multiples and market cap history

16 Data Quality (DQ) Rules Engine:

Validation rules (`src/etl/validator.py`) inspect all incoming data for duplicate primary keys, orphan foreign keys, balance sheet imbalances (Assets != Liabilities), zero sales, OPM divergences > 1%, and non-standard year formats. Any failures are output to `output/validation_failures.csv`.

3. Financial Ratio Engine & Edge-Case Handling

The Ratio Engine (`src/analytics/ratios.py` & `cagr.py`) computes 50+ financial ratios across profitability, leverage, efficiency, cash flow, and growth CAGR metrics.

Core Ratio Formulas & Handling Logic:

KPI Metric	Mathematical Formula	Edge-Case Handling Rules
Net Profit Margin	Net Profit / Sales x 100	Returns None if Sales = 0
Return on Equity (ROE)	Net Profit / (Equity + Reserves) x 100	Returns None if Equity + Reserves <= 0
ROCE	EBIT / (Equity + Reserves + Borrowings) x 100	Returns None if Capital Employed <= 0
Debt to Equity (D/E)	Total Borrowings / Total Equity	Returns 0.0 for debt-free; skipped for Financials in screener
Interest Coverage (ICR)	EBIT / Interest Expense	Returns None and labels 'Debt Free' if Interest = 0
Free Cash Flow (FCF)	CFO - CapEx	Calculated directly from cash flow statement
CFO Quality Score	5-Year Avg CFO / 5-Year Avg Net Profit	Categorised as High (>1.0), Moderate (0.5-1.0), or Accrual Risk (<0.5)

6-Pattern CAGR Edge-Case Taxonomy:

- NORMAL: Positive start and end values $\rightarrow (V_n / V_0)^{1/n} - 1$
- TURNAROUND: Negative start value, positive end value $\rightarrow$ Flagged as TURNAROUND
- DECLINE_TO_LOSS: Positive start value, negative end value $\rightarrow$ Flagged as DECLINE_TO_LOSS
- BOTH_NEGATIVE: Negative start and end values $\rightarrow$ Flagged as BOTH_NEGATIVE
- ZERO_BASE: Zero start value $\rightarrow$ Flagged as ZERO_BASE
- INSUFFICIENT: Less than required years of data $\rightarrow$ Flagged as INSUFFICIENT

4. Screener Engine & Peer Benchmarking

The Stock Screener (`src/screener/engine.py`) provides custom threshold filtering across 15 parameters and 6 predefined strategy presets configured in `config/screener_config.yaml`.

6 Strategy Screener Presets:

Preset Name	Target Investment Style	Key Filter Criteria
Quality Compounder	High-ROE Growth Stocks	ROE >= 15%, D/E <= 1.0, 5Y Rev CAGR >= 10%, FCF > 0
Value Pick	Undervalued Value Stocks	P/E <= 15, P/B <= 2.0, Dividend Yield >= 1.5%
Growth Accelerator	High Revenue/PAT Growth	5Y Rev CAGR >= 15%, 5Y PAT CAGR >= 15%
Dividend Champion	High Payout & Yield	Dividend Yield >= 2.0%, Payout Ratio <= 80%, FCF > 0
Debt-Free Blue Chip	Low Leverage & High Solvency	D/E <= 0.1, ICR >= 5.0, ROE >= 12%
Turnaround Watch	Recovering Profitability	5Y PAT CAGR Flag = TURNAROUND, OPM >= 10%

Peer Percentile Comparison Engine:

The Peer Engine (`src/analytics/peer.py`) categorises all 92 companies into 11 peer groups. It computes percentile ranks (0.00 to 1.00) across 10 core metrics. D/E percentile ranks are inverted so that lower debt yields a higher score. Output is saved to `output/peer_comparison.xlsx` and 90 radar charts in `reports/radar_charts/`.

5. Streamlit Interactive Dashboard Navigation

The Streamlit application (`src/dashboard/app.py`) provides an interactive web UI with 8 screens. All database queries are optimized using `@st.cache_data(ttl=600)` for sub-second page loads.

8 Dashboard Screen Breakdown:

Screen #	Screen File	Primary Purpose & Features
01	01_home.py	Platform Overview, KPI cards, sector market cap distribution, macro stats
02	02_profile.py	Deep-dive 10Y financial statement analysis, P&L, BS, CF, and trend charts
03	03_screener.py	Dynamic stock screener with 6 preset buttons, slider controls, and CSV export
04	04_peers.py	Peer group benchmark table, percentile heatmap, and 8-axis radar overlay
05	05_trends.py	Multi-company 10-year KPI trend comparisons and growth trajectories
06	06_sectors.py	Sector aggregation dashboard, median ROE/ROCE, and constituent rankings
07	07_capital.py	Capital allocation taxonomy, CapEx intensity, and CFO quality analysis
08	08_reports.py	One-click PDF tearsheet download center and batch report generator

6. Cash Flow Intelligence & Distress Detection

The Cash Flow Intelligence module (`src/analytics/cashflow_intelligence.py`) classifies operational cash quality, CapEx intensity, and financial distress signals across all companies.

Capital Allocation Taxonomy & Distress Logic:

• CFO Quality Score: 5Y Avg CFO / Net Profit. Categorized into High Quality (>1.0), Moderate (0.5-1.0), and Accrual Risk (<0.5).

• CapEx Intensity: |CFI| / Sales x 100. Categorized into Asset Light (<3%), Moderate (3-8%), and Capital Intensive (>8%).

• Distress Signal Detection: Triggered when CFO < 0 AND CFF > 0 in the latest financial year. Indicates operational cash burning supported by debt/equity financing.

• Deleveraging Flag: Triggered when CFF < 0 AND total borrowings decrease YoY.

Outputs are saved to `output/cashflow_intelligence.xlsx` (91 rows) and `output/distress_alerts.csv`.

7. NLP Text Parsing & Auto Pros/Cons Generator

The NLP module consists of two engines: text parsing (`src/nlp/parser.py`) and sentiment rule generation (`src/nlp/pros_cons_generator.py`).

NLP Component Architecture:

1. Analysis Text Parser: Uses regex pattern `(\d+)\s*Years?:?\s*([\d.]+)%` to extract period (e.g. 10Y) and values from unstructured text. Saves output to `output/analysis_parsed.csv`.

2. Auto Pros & Cons Generator: Applies 12 Pro Rules (high ROE, positive FCF, debt-free, 5Y CAGR > 15%) and 12 Con Rules (high D/E, negative FCF, falling OPM, high leverage). Assigns confidence scores (60% to 95%). Guarantees at least 1 Pro and 1 Con for every company. Output saved to `output/pros_cons_generated.csv`.

8. Machine Learning KMeans Clustering Engine

The Clustering Engine (`src/analytics/clustering.py`) groups companies into 5 financial archetypes using KMeans machine learning.

5 Financial Archetype Clusters:

Cluster ID	Archetype Cluster Name	Financial Profile & Characteristics
0	High-Quality Compounders	High ROE (>20%), strong OPM, low debt, consistent 5Y revenue/PAT CAGR
1	Defensive Dividend Payers	Moderate growth, high dividend yield, stable CFO, low CapEx intensity
2	Value Cyclicals	Low P/E and P/B multiples, cyclical revenue, moderate leverage
3	Distressed or Turnaround	Negative/low FCF, elevated D/E, contracting margins, turnaround CAGR flags
4	Emerging Growth	High 5Y revenue CAGR (>15%), heavy CapEx reinvestment, moderate margins

Clustering Outputs & Reports:

- `output/cluster_labels.csv`: Master mapping of company_id, cluster_id, cluster_name, and distance_from_centroid
- `reports/elbow_plot.png`: Inertia plot for $k=2 \dots 10$ confirming $k=5$ near the elbow point
- `reports/correlation_heatmap.png`: Pearson correlation matrix across 10 core KPIs
- `output/outlier_report.csv`: Outlier detection flagging companies with Z-score > 3 per broad sector
- `output/portfolio_stats.csv`: P10, P25, P50, P75, P90, Mean, and Std metrics across 92 companies

9. REST API Server & Endpoint Developer Reference

The FastAPI server (`src/api/main.py`) exposes 16 RESTful HTTP endpoints returning JSON data and PDF downloads. Server starts via `uvicorn src.api.main:app --port 8000`.

16 Live API Endpoints Reference:

HTTP Method	Endpoint Path	Description & Response Content
GET	/api/v1/health	System health check, uptime, and database row counts
GET	/api/v1/companies	List all 92 companies with sector mapping & filter params
GET	/api/v1/companies/{ticker}	Full company profile & latest year financial ratios
GET	/api/v1/companies/{ticker}/pl	Income statement history array
GET	/api/v1/companies/{ticker}/bs	Balance sheet statement history array
GET	/api/v1/companies/{ticker}/cashflow	Cash flow statement history array
GET	/api/v1/companies/{ticker}/ratios	Multi-year computed financial ratios array
GET	/api/v1/companies/{ticker}/tearsheet	Binary 2-page PDF tearsheet download
GET	/api/v1/screener	Stock screener endpoint with min_roe, max_de, min_fcf params
GET	/api/v1/sectors	Sector summary list with company count and median ROE
GET	/api/v1/sectors/{sector}/companies	Constituent companies for a given broad sector
GET	/api/v1/peers/{group_name}	Peer group members and benchmark indicator
GET	/api/v1/companies/{ticker}/peers/compare	8-axis radar comparison data vs peer group avg
GET	/api/v1/market-cap/{ticker}	Historical market cap & valuation multiples (P/E, P/B)
GET	/api/v1/portfolio/stats	P10 through P90 percentile distribution for 10 KPIs
GET	/api/v1/companies/{ticker}/documents	Annual report PDF links with URL validation status

10. Makefile Target Commands & Troubleshooting

The platform includes standardized `Makefile` targets for executing ETL pipelines, ratio calculations, test suites, dashboard UI, and API servers.

Makefile Target Commands Reference:

Command Target	Description & Pipeline Execution Details
make load	Executes loader.py: Ingests 12 Excel files, validates 16 DQ rules, populates nifty100.db
make ratios	Executes engine.py & ratios.py: Computes 50+ KPIs for 1,155 company-years
make test	Executes pytest suite (123 tests) and generates reports/pytest_report.html
make report	Generates 91 company tearsheet PDFs, 10 sector PDFs, and portfolio summary PDF
make dashboard	Launches Streamlit 8-screen dashboard on http://localhost:8501
make api	Launches FastAPI REST server on http://localhost:8000 (OpenAPI docs at /docs)
make clean	Removes bytecode caches (.pyc) and temporary test artifacts. Database remains intact.

Troubleshooting & System FAQs:

1. SQLite Database Lock Errors: Ensure no external SQLite viewers are locking `data/nifty100.db`. Close active DB Browser connections before running `make load`.

2. Port Conflicts: Streamlit defaults to port 8501 and FastAPI to port 8000. If port is in use, kill existing process or pass `--port `.

3. PDF Overflow: All tearsheet tables use flowable wrappers and explicit cell widths to prevent text overflow. Verify reportlab version is >= 3.6.